Le metriche del software

Definizione

- Le metriche del software
  - Strumenti utili per misurare e valutare caratteristiche del software
  - Componente essenziale dell'ingegneria
    - Non si può controllare ciò che non si può misurare
  - Numeri assegnati a proprietà specifiche del software
    - Significato ben definito
  - Utilizzate per diverse finalità
    - Conoscere i costi del software
    - Valutare la produttività dei programmatori
    - Stimare la qualità del codice prodotto
    - Identificare opportunità di miglioramento della qualità
- Principali aree misurate
  - **Stima dei costi e dei tempi necessari per lo sviluppo del software**
  - **Produttività dei programmatori**
    - Valutazione basata sul lavoro svolto e il tempo impiegato
  - **Dimensione del software**
    - Misurata in linee di codice sorgente o occupazione di memoria e risorse
  - **Funzionalità del software**
    - Valutazione del grado di completezza delle funzioni implementate
  - **Complessità del software**
    - Misurata in base alla struttura e all' interconnessione delle componenti
  - **Modularità ed evolvibilità del software**
    - Valutazione della facilità di modificare o estendere il sistema

Misurazione

- Processo di assegnazione di simboli, principalmente numeri, per rappresentare gli attributi delle entità di interesse
  - Identificazione dell'entità (persona, viaggio, casa, ...)
  - Attributo da misurare: caratteristica dell' oggetto (altezza, durata, costo, ...)
  - Forma di rappresentazione
    - Altezza si misura in centimetri
    - Indumenti con taglie: S (small), M (medium), L (large), XL (extra large)
  - Regole per ottenere una misura coerente, ripetibile, non soggettiva

Metriche

- Misurazione delle metriche
  - **Misurazioni dirette e indirette**
    - Misure dirette basate su strumenti specifici (lunghezza, volume)
    - Misure indirette combinano misure dirette per attributi non direttamente misurabili
    - Esempio di misura indiretta nel decathlon
      - Misurazione di tempi e lunghezze
      - Combinazione e ponderazione delle misure per ottenere un "punteggio" identificativo del "miglior atleta"
    - Misura degli attributi, non delle entità
      - Esempio: misurare l'altezza di Franco, non Franco stesso
- **Metriche dimensionali del software**
  - Misurano la dimensione del software, non necessariamente la complessità
    - Quindi è necessario utilizzare ulteriori metriche e valutazioni per valutare la complessità e la qualità complessiva del software
  - **Metriche dirette del software**

LOC	Numero di linee di codice
KLOC	corrisponde a 1000 LOC
ES	Numero di istruzioni eseguibili e dichiarative
CLOC	Numero di righe di commento
NCLOC	Numero di righe senza commenti
CP	Percentuale di commenti
Occupazione di memoria del codice oggetto (in Byte)	
Occupazione di memoria del codice sorgente (in Byte)	
Numero di caratteri del testo del programma (CHAR)	CHAR = a * LOC, dove a è il numero medio di caratteri per riga di codice

- **Altre metriche dirette**
  - **Durata del progetto (giorni)**
  - **Durata dell'attività (giorni o ore)**
  - **Sforzo del progetto (mesi, giorni o ore-uomo)**
  - **Numero di errori trovati durante il testing (BUG)**
  - **Numero di cambiamenti apportati al programma in un anno (ACT)**
- **Esempi di metriche indirette**

Produttività (P):	LOC prodotte / unità di tempo (LOC/giorno)
Qualità (Q):	LOC / numero di errori rilevati durante il testing
Densità di errori (I/LQ):	rapporto tra il numero di errori e le linee di codice
Livello di documentazione (D):	numero di pagine di documentazione (PD) / LOC
Stabilità dei requisiti:	rapporto tra il numero iniziale e il numero totale di requisiti nel tempo

- **Metriche funzionali**
  - **Analisi dei Punti Funzione (FPA - Function Point Analysis)**
    - Valutazione della dimensione delle funzionalità e stima del costo di implementazione dei requisiti funzionali
- Selezione e utilizzo delle metriche appropriate
  - Basato sugli obiettivi specifici e le caratteristiche del progetto software
  - Per ottenere una valutazione accurata delle diverse dimensioni e qualità del software.

Function Points

- Metrica per misurare le funzionalità richieste dall'utente e la dimensione dello sviluppo o della manutenzione del software
- Caratteristiche principali dei Function Points
  - Indipendenza dalla tecnologia
    - Misura coerente tra progetti, linguaggi di programmazione e produttori diversi
  - Coerenza tra progetti
    - Misura standardizzata per il confronto e la valutazione comparativa della dimensione e della produttività del software
  - Misurazione di diversi linguaggi
    - Concentrazione sulla funzionalità richiesta, indipendentemente dalla specifica implementazione tecnica
- IFPUG (International FP User Group)
  - Principale associazione internazionale per standardizzare i Function Points
- **Function Types utilizzati nei Function Points**
  - **External Inputs (EI)**
    - Numero di input distinti forniti dall'utente al sistema
    - **External Outputs (EO)**
      - Numero di output distinti prodotti dal sistema come risultato delle elaborazioni
      - **External Inquiries (EQ)**
        - Numero di richieste dell'utente che richiedono una risposta immediata dal sistema
        - **Internal Logical Files (ILF)**
          - **Numero di file creati e utilizzati internamente dal sistema**
          - **External Interface Files (EIF)**
            - **Numero di interfacce esterne, come file o altri insiemi di dati scambiati dal programma con altri programmi**
            - **Identificazione EIF**
              - Regole per l'identificazione di un EIF
                - **EIF è un gruppo di dati logico identificabile dall'utente, esterno all' applicazione**
                - Il gruppo di dati non gestito da parte dell' applicazione misurata
                - Il gruppo di dati è conteggiato come un ILF per un'altra applicazione.
                - Il gruppo di dati identificato non è conteggiato come un ILF per l'applicazione corrente.
              - Elementi considerati come EIF
                - File ricevuti da un'altra applicazione
                - Archivi dei messaggi di help
            - **Identificazione ILF**
              - Regole per l'identificazione di un ILF
                - **ILF è un gruppo di dati logico identificabile dall'utente**
                - Gestione del gruppo di dati da parte di un processo interno dell'applicazione
              - Elementi considerati come ILF
                - Entità logiche dei dati rilevanti per l'utente
                - Struttura, classi, tabelle o file rappresentanti gli impiegati
                - File o tabelle interne generate o gestite dall' applicazione
                - File o tabelle gestite dall'utente
              - Elementi non considerati come ILF
                - File intermedi o di sort
                - File di cui l'utente non ha visibilità
- Combinazione dei Function Points con altre metriche
  - Numero di linee di codice (LOC)
  - permette l'ottenimento di informazioni sulla produttività, qualità e complessità del software sviluppato
- **Complessità di un ILF / EIF**
  - Calcolo della complessità
    - Dopo aver identificato gli ILF e gli EIF, è necessario calcolarne la complessità tenendo conto del numero di RET e DET.
    - **RET (Record Element Type)**
      - Rappresenta un record riconoscibile dall'utente in un ILF o EIF
      - Esempi: Descrizione di un impiegato, fattura
    - **DET (Data Element Type)**
      - Rappresenta un campo riconoscibile dall'utente
      - Ad esempio, il nome, lo stipendio o l'indirizzo di un impiegato
    - Regole per il conteggio di RET e DET
      - Senza sottogruppi: ILF o EIF composto da uno o più RET
      - DET per ogni campo utilizzato dall'utente per stabilire una relazione con un altro ILF o EIF
  - **Tabella di complessità**
    - Livelli di complessità: bassa, media, alta

	1..19 DET	*20..50 DET*	*51 + DET*
1 RET	Bassa	Bassa	Media
2..5 RET	Bassa	Media	Alta
5+ RET	Media	Alta	Alta

  - Analogie con la complessità di EI, EO, EQ
    - Utilizzo di un metodo simile per valutare la complessità

Metriche Object-Oriented

- Metriche Object-Oriented
  - Sistemi orientati agli oggetti
    - Differenti dai sistemi procedurali
    - Basati sul concetto di classi che combinano dati e metodi
    - Caratteristiche peculiari dei sistemi orientati agli oggetti
      - Incapsulamento
      - Occultamento dell'informazione
      - Ereditarietà
      - Composizione
      - Messaggi
  - Metriche OO per le classi
    - Metriche di dimensione a livello di sistema
      - Line of Code (LOC)
      - Numero di classi
      - Numero di metodi nel sistema
      - Valori medi di metriche calcolate sulle classi per una visione complessiva delle dimensioni del sistema
    - Metriche di dimensione di classe
      - LOC della classe
      - Numero di metodi della classe
      - Alcuni casi: Peso dei metodi basato sulla complessità ciclomatica (VG3)
- Metriche di Chidamber e Kemerer (Metriche CK)
  - Introdotte da Chidamber e Kemerer nel 1994
  - Metriche strutturali di progetto
  - Applicabili al progetto dettagliato del sistema o al codice sorgente
  - Le sei metriche CK:
    - **DIT (Depth of Inheritance Tree)**
      - Misura la profondità dell'albero di ereditarietà
      - Rappresenta la distanza massima di una classe dalla radice dell'albero delle eredità
      - La radice ha DIT = 0
      - Nel caso di ereditarietà semplice, il valore di DIT corrisponde al numero di superclassi presenti
      - Più una classe è distante dalla radice, maggiore è la possibilità che erediti variabili e metodi da classi superiori
      - Una classe con un alto valore di DIT è più complessa e difficile da comprendere
      - Si consiglia di mantenere il valore di DIT entro 5-6 per evitare eccessiva complessità
    - **NOC (Number of Children)**
      - Misura il numero di figli diretti di una classe
      - Indica il numero di sottoclassi immediate di una determinata classe
      - Se una classe è una foglia, il valore di NOC è 0
      - All'aumentare di NOC, aumenta il livello di riuso della classe
      - Si corre il rischio di incorrere in astrazioni errate con un alto valore di NOC
      - Un valore elevato di NOC comporta un maggior numero di test necessari per verificare i metodi della classe
      - In generale, il valore di NOC dovrebbe rimanere entro 10-15, a meno di casi particolari
    - **WMC (Weighted Methods per Class)**
      - Misura la complessità della classe basata sul numero di metodi ponderato
      - Rappresenta la somma dei pesi dei metodi di una classe
      - Il peso di ciascun metodo può essere misurato, ad esempio, dalla sua complessità ciclomatica
      - In generale, WMC = Σ c_i, dove c_i è il peso di ciascun metodo
        - Spesso si assume $c_i = 1 \Rightarrow$ WMC = N, corrispondente al numero totale di metodi nella classe
        - Di solito non tiene conto dei metodi ereditati
        - È importante trovare un equilibrio nella quantità di metodi nella classe per mantenere la coesione
    - **LCOM (Lack of Cohesion of Methods)**
      - Misura la mancanza di coesione dei metodi nella classe
      - Rappresenta il numero di coppie di metodi di una classe che condividono una o più variabili di istanza
      - Se non ci sono metodi che accedono alle stesse variabili, il valore di LCOM è massimo
      - LCOM è calcolo come la differenza tra il numero di coppie di metodi che non condividono alcune variabile (P) e il numero di coppie di metodi che condividono almeno una variabile (Q)
      - Se P < Q, allora LCOM = 0
      - LCOM è un indicatore della coesione della classe: un valore più basso indica una maggiore coesione
    - **RFC (Response for a Class)**
      - Misura l'numero di metodi che possono essere chiamati in risposta a un messaggio inviato a una classe
      - Rappresenta l'insieme dei metodi che possono essere eseguiti in risposta a un messaggio ricevuto da un oggetto della classe
      - Include i metodi definiti nella classe stessa, inclusi quelli ereditati, e i metodi definiti in altre classi che vengono chiamati all'interno dei metodi della classe
      - Misura la complessità della classe e la sua capacità di comunicare con altre classi
      - Un valore più alto di RFC rende la classe più difficile da capire, modificare e correggere
    - **CBO (Coupling Between Object Classes)**
      - Misura l'accoppiamento di una classe con altre classi
      - Rappresenta il numero di classi a cui una classe è accoppiata
      - Considerando le collaborazioni, CBO è il numero di classi utilizzate dalla classe in questione
      - Indica il numero di classi da cui dipende o che dipendono dalla classe (contate una sola volta)
      - Un valore più alto di CBO indica una maggiore possibilità che la classe sia influenzata da eventuali modifiche alle classi dipendenti
      - Un elevato valore di CBO rende la classe più difficile da comprendere, modificare e correggere

Complessità ciclomatica di McCabe

- **Rappresenta la complessità logica di una funzione**
  - **Ossia lo sforzo richiesto per implementarla e comprenderla**
  - Calcolata utilizzando il flusso di controllo della funzione
    - Flusso di controllo rappresentato da un grafo G
- **Formula per il calcolo della complessità ciclomatica V(G)**
  - **V(G) = A - N + 2P**
    - A: numero di archi nel grafo G
    - N: numero di nodi nel grafo G
    - P: numero di sottografi disconnessi (di solito P = 1)
- **Complessità ciclomatica corrisponde al numero di decisioni più uno**
  - Decisione rappresenta la scelta tra un ramo e un altro nel flusso di controllo
- Utilità del CC di McCabe
  - **Guida per identificare parti di codice complesse**
    - Valore elevato suggerisce la necessità di suddividere la funzione o semplificare la logica condizionale
  - che richiedono attenzione durante sviluppo, testing e manutenzione
- Importanza della linearità del flusso di controllo
  - CC di McCabe inferiore è preferibile
    - Indica maggiore linearità nel codice
    - Facilita la comprensione e la gestione del codice
- Valutazione combinata con altre metriche e considerazioni specifiche del contesto del progetto

Cyclomatic Complexity	Risk Evaluation	Probability of Bad fix
1-10	Low risk, testable code	5%
11-20	Moderate Risk	10%
21-50	High Risk	30%
>50	Very High Risk, untestable code	40%

Tabella Unadjusted Function Points

- ■ **ILF** Bassa 7
  ■ Media 10
  ■ Alta 15
- ■ **EIF** Bassa 5
  ■ Media 7
  ■ Alta 10
- ■ **EI** Bassa 3
  ■ Media 4
  ■ Alta 6
- ■ **EO** Bassa 4
  ■ Media 5
  ■ Alta 7
- ■ **EQ** Bassa 3
  ■ Media 4
  ■ Alta 6
- **Il numero di Function Points (FP) è la somma dei punti di complessità di tutti i tipi di funzioni trovate**
- Evoluzione della FPA
  - Varianti dei Function Points
    - OO Function Points (Object Points)
      - Effettuano un mapping diretto delle classi del modello a oggetti del sistema sugli elementi costituenti il calcolo dei FP
  - **Use Case Points**
    - Basati sul diagramma dei casi d'uso
    - Conteggio di Use Case e attori
    - Peso degli attori in base alle interazioni con il sistema
    - Peso dei casi d'uso in base al numero di transazioni
    - Somma pesata di attori e Use Case
    - Possibilità di applicare correttori per la complessità delle funzioni
  - **SNAP (Software Non-functional Assessment Process)**
    - Metodo IFPUG per stimare lo sforzo dei requisiti non funzionali

Presented with xmind